

Improved APIO: Automatic Prompt Induction and Optimization Method Improvement via Genetic Algorithm

Tran Thanh Thao¹, Nguyen Khanh Linh¹,
Nguyen Huu Phu¹, Hoang Thien Truong¹

¹University of Engineering and Technology - Vietnam National
University, Hanoi, Vietnam.

Contributing authors: 22021157@vnu.edu.vn; 22021158@vnu.edu.vn;
22021138@vnu.edu.vn; 22021219@vnu.edu.vn;

Abstract

Automatic Prompt Optimization (APO) plays an important role in improving the performance of large language models in language processing tasks. APIO is a typical approach for this problem, based on two stages: prompt induction and prompt optimization, which automatically improve instructions for each task. However, the original APIO still has some limitations, including the use of world level metrics, a prompt induction mechanism with only a single agent, to name a few.

To overcome the above limitations, this research proposes an improved version of APIO that uses a multi agent mechanism in the prompt induction step, as well as cosine similarity, a semantics based metrics in the optimization process, while also applying a genetic algorithm based optimization strategy. The method was evaluated on two tasks: Grammar Error Correction (GEC) and text simplification, with better results in both efficiency and stability compared to the original APIO method.

Keywords: APO, Prompt Optimization, Genetic Algorithm, GEC, Text Simplification

1 Introduction

Automated Prompt Optimization, or APO, are methods used to automate prompt optimization by systematically suggesting improvements to improve the results received from LLM, while reducing human’s work in the process. APIO is a new but promising method in this area. Its basic idea is to combine two stages: prompt induction and prompt optimization, the details of which will be in below sections.

However, the original APIO still has some problems, some of which are as follow. First, the authors use Levenshtein metrics to evaluate outputs, which is a word-level metrics, while the quality of the prompt leans more towards meaning. Second, the original paper only uses one AI agent, so the generated instruction can be biased or lack diversity. Finally, in the prompt optimization step, the chosen candidates used for validating may be overfitted and does not necessarily represent the entire dataset or the entire language, so the real-world uses may be limited.

Thus, this paper focuses on improving the original APIO to enhance the efficiency and adaptability of the method by solving the above problems.

2 Original Method (APIO)

The original method [1] has two main steps: Prompt Induction and Prompt Optimization.

Prompt Induction

At first, APIO uses a number of input-output pair, taken from the training data, that represents what the task is like. These examples are sent to a state-of-the-art model (GPT-4.1-mini in this case) to infer a prompt to solve the corresponding task so that the output can be achieved from the input.

An important feature in this stage is the structured instructions, namely the mark-down list, where each item is a single instruction. To be specific, after this step, an instruction list L is produced, where each instruction is the prompt result of a input-output pair. This approach allows for fine-tuning at the instruction level, providing greater control to each instruction than an entire paragraph.

Prompt Optimization

In this step, the authors iteratively optimize the induced prompt from the previous step through three steps:

1. The instruction pool is first initialized to L , the set of instructions inferred during the Prompt Induction step
2. Then the authors expand the above pool through Beam Search with a beam size B . In particular, they expand the pool through three prompting operations:
 - **Improve:** They generate beam candidates by prompting an LLM to improve the given pool of instructions to reduce the error rate on the given input-output

examples as much as possible, using Levenshtein edit distance as a metric for optimization for all domains

- **Rephrase:** Expand the current pool by prompting the LLM to rephrase each instruction without changing the underlying meaning
- **Permute:** Take N_{permute} instructions and randomly change their order in the current list of instructions

3. After expanding the pool using above operations, APIO obtain three candidate sets - each set being a list of instructions, which are ranked by their performance on the validation set and add the best B to the pool. To control divergence from prior iterations, the authors additionally introduce a word level Levenshtein edit distance penalty on the prompts.

3 Proposed Method

As stated in the previous section, the original APIO paper proposes some problems, so we aim to reduce model bias and avoid overfitting in the process by improving on some points.

The general idea of the proposed method [2] is to combine using Cosine-level similarity (a meaning-based metrics), applying a multi agent mechanism, and using selection strategies based on genetic algorithms to improve the quality and stability of the optimized prompt. The output is the most optimal prompt, achieving high and stable performance according to the defined evaluation criteria.

After our improvement, the method can be described through the following steps. First, the initial prompts are induced using a multi agent mechanism. Then a set of candidate prompts are created through several prompt altering steps like improvement, rephrasing and permutation, to enlarge the set. They are then evaluated using Cosine metrics on the embedding vector. Afterwards, the prompts are selected using a combination of mean and weighted random selection strategies to allow more natural selection. Finally, the optimization process is repeated over multiple rounds until the best prompt is selected.

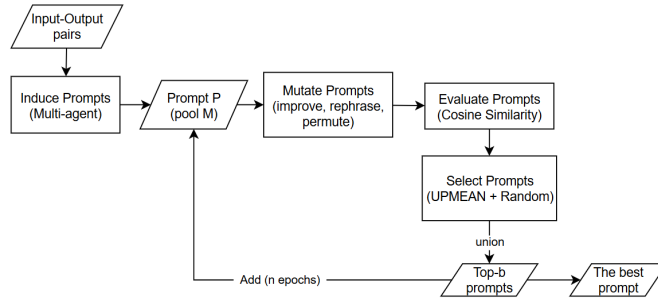


Figure 1: Overview of Improved APIO

3.1 Replacing Levenshtein distance evaluation with Cosine Vector Embedding

In the original method, at the Improve step of the prompt optimization phase, the authors used the Levenshtein correction distance at the word level to assess the quality of the instruction set. After applying the instruction set to the input data and generating the corresponding output, this output was compared with the standard output in the training set. If the difference in word-level between the two outputs is too big, meaning the Levenshtein distance was large, the instruction set would be penalized.

The limitation of this approach is that Levenshtein only shows word-like difference, while many tasks require meaning-based approach. Two sentences may have the same meaning but completely different words, making Levenshtein distance ineffective for such tasks.

So, we propose replacing Levenshtein with another method based on meaning, namely cosine similarity. To explain, each sentence is encoded into a space vector, representing both lexical and semantic information. Cosine similarity between the generated output and the standard output is used to evaluate, and instruction sets that produce output with low score are removed. This approach allows to judge the meaning of the output, thus improving the reliability of the optimization phase.

3.2 Implementing multi agent Model in Prompt Induction Phase

Currently in the original method, the authors use only a single model for the prompt induction step, making the results biased by the opinion of a single model. Therefore, we adopt a multi agent approach, where additional LLMs are added to act as evaluators and provide feedback to improve the quality of the generated instructions.

Specifically, our process uses three advanced LLMs: one model acting as the student, another acting as the teacher (providing feedback), and a third model acting as the editor. The initial instruction created by the student model is passed to the teacher model for evaluation and feedback. Both of them are then passed to the editor agent, where the instruction is rewritten based on the teacher’s suggestions, mimicking the feedback and improve process in the real world.

This approach adds a layer of feedback and evaluation to the prompt induction process, helping to reduce bias due to errors caused by a single model and producing more stable and reliable instructions.

3.3 Implementing Genetic inspired Algorithm in APIO process

To reduce the risk of overfitting caused by randomly selecting validation set, we use a algorithms based on genetics algorithm at the beam search stage of the Prompt Optimization phase. Our approach is as follows.

We treat the instruction set generated during prompt induction as the initial parent population. The Improve, Rephrase, and Permute operations then act as mutation operators, producing successive generations of candidate instruction sets, essentially omitting the crossover step.

The candidates are then evaluated on the validation set to compute a Validation Score for each instruction set. Instead of selecting the top B candidates only based on this score, we computed the mean Validation Score and selected all candidates performing above this threshold to form set A.

At the same time, we randomly sample n candidates from the entire candidate pool. The candidates are chosen based on their validation scores, meaning while it may be random, the candidates with higher validation scores are still more likely to be selected. We put them into set B.

We then pick the similar candidates from both set A and B to create a population for the next iteration. Then we round the set size to match the beam size, meaning if the size is more than the beam size, we remove the weakest candidates and if it is fewer, we pick additional candidates from both sets. This is repeated at each iteration.

This approach reduces the overfitting, which is a problem present in the original method, while still ensuring that candidates that don't perform too well at the Optimization phase still stands a chance to survive and improve later, better supporting generalization to real world usage.

4 Experimental Results

4.1 Configuration

4.1.1 Benchmark

We used the existing datasets: BEA-2019 [3] for Grammar Correction and ASSET [4] for Text simplification, which was used in previous researches.

Dataset	Purpose	Size
BEA-2019	Evaluation of grammatical error correction capability of the APIO method.	4,477 sentences
ASSET	Evaluation of sentence rewriting capability of the APIO method.	2,359 original sentences, each with 10 corresponding rewrites

Table 1: Datasets used for evaluating the APIO method

4.1.2 LLM Model

The paper uses GPT 4.1 mini, a model known for the balance between performance and cost. Even though the model is smaller in comparison to state of the art models

like GPT 5.0, it is still capable of handling contexts up to 33000 tokens, more than enough for most GEC and Text Simplification tasks. The model also has a knowledge cutoff since June 2024, but still remains effective for the intended applications.

4.1.3 Baselines

Since this paper aims to improve upon the existing APIO method, the baseline is established using APIO with the parameters shown in the table below.

```

1 # Configuration for instruction optimization
2 hyperparameters = {
3     "beam_size": 8,
4     "num_optimization_iterations": 1,
5     "num_rephrased_samples": 3,
6     "rephrase_scale_factor": 2,
7     "permuted_instructions_scale_factor": 1,
8     "improve_by_feedback_scale_factor": 1,
9     "training_batch_size": 4,
10    "max_num_instructions": 15,
11    "num_instruction_feedback_trials": 3
12 }
```

Listing 1: Model Hyperparameters

Each time we run, the beam size and the number of optimization iterations are changed to see their impact on the method’s performance. All other parameters are kept the same to create a fair and consistent comparison between the original and improved approach.

4.1.4 Metric

The study uses two metrics: F0.5 for grammatical error correction and SARI for text simplification.

F0.5 is a very popular metric in grammatical error correction and serves as the official evaluation standard for major competitions in this field like CoNLL 2014 and BEA 2019. It is also a mandatory metric to be reported in nearly all studies focusing on GEC.

$$F_{0.5} = (1 + 0.5^2) \frac{P \cdot R}{(0.5^2 \cdot P) + R} = 1.25 \frac{P \cdot R}{0.25P + R}$$

SARI is another commonly used metric in text simplification research and is currently the main measure for datasets such as ASSET and TurkCorpus. It has replaced BLEU in this domain because BLEU is not suitable for evaluating simplification tasks, like text simplification.

$$\text{SARI} = \frac{1}{3} (F_{\text{add}} + F_{\text{keep}} + P_{\text{del}})$$

4.2 Answers to Research Questions

4.2.1 Answer to RQ1 - Prompt Optimization Effectiveness: Impact of replacing Levenshtein with cosine similarity in GEC and text simplification

Task	Beam_size	Loop	APIO (original)		APIO (Cosine Only)	
			init_score	best_score	init_score	best_score
GEC	2	1	45.23	46.15	32.17	41.32
	4	1	46.15	47.81	40.85	41.25
	6	1	48.34	48.63	34.22	41.8
	8	1	46.73	50.75	43.73	43.97
Text Simplification	2	1	49.83	50.16	49.92	50.94
	4	1	49.77	50.01	49.62	50.08
	6	1	49.89	50.35	49.5	49.8
	8	1	49.63	49.78	49.54	50.95

Table 2: Performance comparison between original APIO and Cosine only APIO.

As seen from the table, the Cosine version shows a large but inconsistent improvement. With the original APIO, the improvement from init score to best score is quite stable and positive, as seen in the last line which increased from 46.73 to 50.75, approximately 4 points. As for APIO Cosine, there were some very large jumps, such as the increase from 32.17 to 41.32, an increase of more than 9 points, or another increase from 34.22 to 41.8. It can be concluded that the surge in the Cosine version is actually due to the very low starting point (32.17, 34.22). The system easily found a better prompt than the bad one initially, but even after optimization, it still didn't reach the performance point of the original algorithm.

The reason for this is due to the nature of the grammar correction problem, which requires accuracy in terms of characters and vocabulary. Two sentences may have very similar meanings, but one is grammatically correct and the other is not. The Levenshtein metric in the original APIO allows for the changes in word level, helping prompts produce output that closely matches the original but corrects minor errors, instead of prompts that produce sentences with the same meaning but significantly altered structure, a problem that cosine metric faces.

In contrast to the GEC task, replacing the Levenshtein metric with Cosine Similarity in the Text Simplification task produces positive results. The Cosine Similarity method performs well and is stable, even slightly better than the original method. Looking at the best score, with the original APIO, the best score only fluctuates around 49.78 - 50.35, while with the Cosine APIO, the best score is higher, peaking at 50.94 and 50.95.

4.2.2 Answer to RQ2 - Prompt Induction Stability: Impact of multi agent mechanisms versus single-agent approaches on instruction quality

Task	Beam Size	Loop	APIO (Original)		APIO (Multi-Agent Only)	
			init_score	best_score	init_score	best_score
GEC	2	1	45.23	46.15	44.00	46.83
	4	1	46.15	47.81	45.20	45.79
	6	1	48.34	48.63	44.57	48.36
	8	1	46.73	50.75	43.13	48.80
Text Simplification	2	1	49.83	50.16	50.02	50.79
	4	1	49.77	50.01	50.49	50.74
	6	1	49.89	50.35	50.18	50.58
	8	1	49.63	49.78	50.36	50.40

Table 3: Performance comparison between original APIO and multi-agent only APIO.

When comparing the init score column between the two tables, it’s clear that the original APIO has a high initial score (45.23 - 48.34), while the APIO (multi agent) has a relatively lower initial score (43.13 - 45.2). Instead of improving the initial prompt, these agents seems to made the prompt worse compared to the simple initialization step of the original APIO.

The most likely cause is that the teacher agent gave negative feedback on a previously good prompt, such as "lacking creativity" or "needs more details," and then instructed the editor agent to rewrite it. As a result, the new prompt became complex, leading to more errors in the correction and a drop in the F0.5 score.

This shows that for tasks requiring high accuracy and have little room for creativity, such as GEC, the judgments of the LLM Agents do not align with the evaluation metrics, in this case, F0.5. Putting a feedback model to change the prompt before experimental results are available tends to alter the prompt’s original goal.

Unlike GEC, in the Simplification problem, the multi agent mechanism proved effective in the instruction generation step. It can be seen that the multi agent APIO produced better instructions, resulting in a significantly higher init score than the original APIO. The multi agent APIO’s initial score was very high, frequently exceeding 50.0, while the original APIO was only at 49.6-49.8. Thanks to the good influence from the multi agent approach, the experiment achieved significantly higher best scores, up to 50.79, about 0.4 points higher than the original APIO.

4.2.3 Answer to RQ3 - Adaptability of the Proposed Method: Differential effects of APIO improvements on GEC and text simplification.

Task	Beam Size	Loop	APIO (Original)		APIO (All)	
			init_score	best_score	init_score	best_score
GEC	2	1	45.23	46.15	30.00	46.49
	4	1	46.15	47.81	44.62	47.50
	6	1	48.34	48.63	46.38	48.63
	8	1	46.73	50.75	50.75	50.91
Text Simplification	2	1	49.83	50.16	50.31	50.41
	4	1	49.77	50.01	50.06	50.40
	6	1	49.89	50.35	50.72	50.72
	8	1	49.63	49.78	50.73	50.88

Table 4: Performance comparison between original APIO and fully improved APIO.

Experimental results show a contrasting effect of the Cosine Similarity and multi agent technical improvements when applied to two tasks of opposite nature: grammar correction and text simplification.

Regarding the impact of changing the metric, in the grammar correction task, replacing Levenshtein with Cosine Similarity significantly reduced performance, while in the simplification task, it improved performance, achieving a higher maximum score. This can be explained by the difference in the nature of the two tasks. Grammar correction is a task with a hard constraint: correcting errors while preserving the original sentence structure, while simplification is a problem with soft constraints, aiming to rewrite sentences to make them more understandable. So the cosine similarity allows the model to freely transform sentences as long as the meaning remains the same, making it suitable to the simplification task, while causing a heavy F0.5 penalty for altering sentence structure.

On the other hand, with the multi agent mechanism, it can be seen that for grammatical error correction, this mechanism reduces the init score, although the subsequent optimization is very good, it is still not enough to compensate for the initial loss. In contrast, the simplification problem uses a multi agent mechanism to improve the quality of instructions in the initialization step, laying the foundation for optimization, thereby producing a final result better than the original APIO method.

This is due to the difference in evaluation. In the grammar correction problem, the critique agent tends to evaluate based on natural language processing, while the scoring metrics for this problem is ERRANT/F0.5, which is a machine-compliant tool. The critique agent will then do the opposite of F0.5; while one considers an

instruction good, the other finds it bad, and vice versa. In the simplification task, the scoring metrics is SARI, which is based on qualitative output answers generated by humans, which is the strength of the initializing agent.

5 Discussion

The most significant factor to be considered in the paper is its reliance on the large, closed source GPT 4.1 Mini. Because the workings and training data of the model are not publicly available, the study faces the risk of changes to the model over time and potential data leaks, making it difficult to reproduce results one-to-one. However, since the focus is on evaluating the prompt optimization method, the value of the paper remains the same even if the underlying model changes.

Intrinsic factors including the randomness of the LLM and parameter sensitivity may also affect the results. The paper controlled for this by setting the temperature to 0 during evaluation and using multiple sampling and averaging the results during the optimization phase.

6 Conclusion and Future Work

In this paper, we presented three improvements to the original APIO method, solving serious problems in both the Prompt Induction and Prompt Optimization phases. These improvements lead to significantly better results across all runs with different beam sizes and loops, as shown in the experiment results.

For future work, our current code uses GPT-4.1-mini, which is quite costly, especially when running multi-agent. This can be reduced by using open-sourced models like Qwen or Llama, which can be good enough with fine-tuning.

Another thing to improve is the optimization of beam size and the loops. Currently, these are manually set, which may lead to bad performance.

References

- [1] Chernodub, A., Saini, A., Huh, Y., Kulkarni, V., Raheja, V.: Apio. arXiv preprint arXiv:2508.09378 (2025)
- [2] Hoang, T.T.: Improved APIO. <https://github.com/thChg/APIO>
- [3] Christopher Bryant, E.A.T.B. Mariano Felice: BEA-2019. <https://aclanthology.org/W19-4406/>
- [4] Fernando Alva-Manchego, A.B.C.S.B.S.L.S. Louis Martin: ASSET Test. <https://aclanthology.org/2020.acl-main.424/>